AI ASSISTED CODING

Assignment-13.1

Name: G.Laxmi Pranav

HT.NO:2303A52403

Batch:36

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

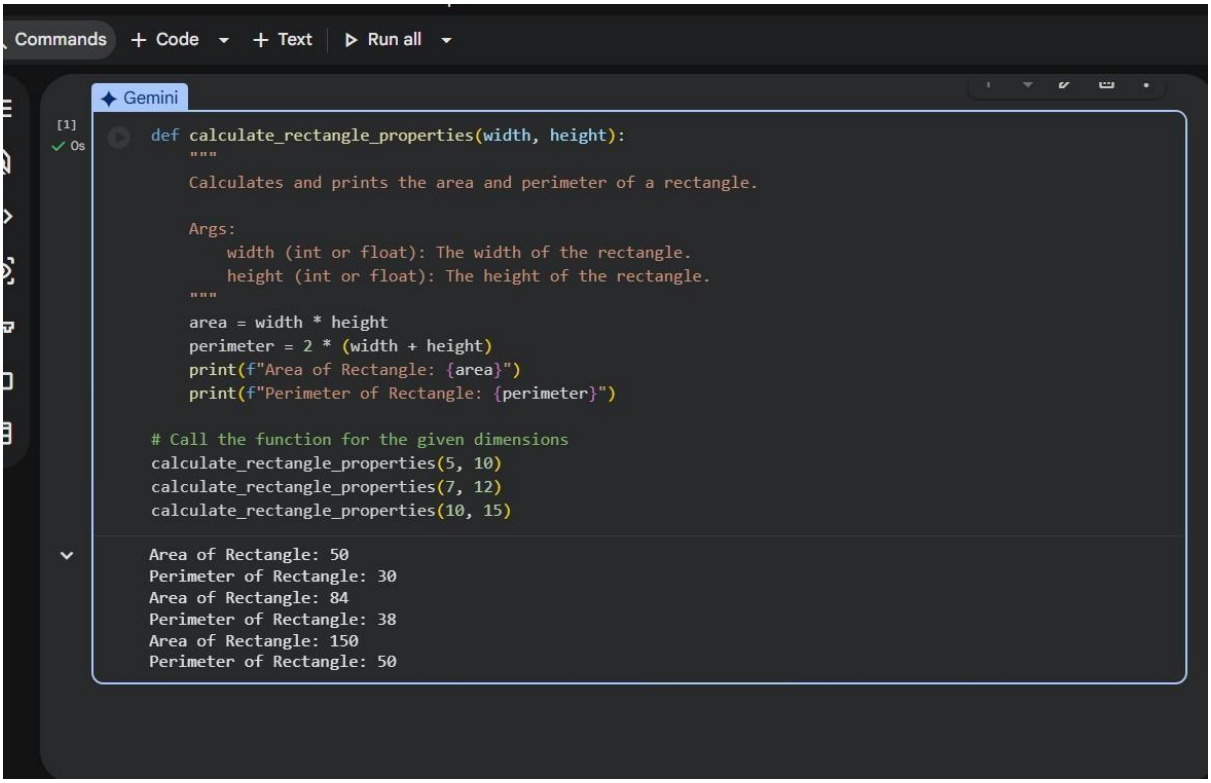
• Instructions: o Prompt AI to identify duplicate logic and replace it with functions or classes. o Ensure the refactored code maintains the same output. o Add docstrings to all functions.

• Sample Legacy Code:

```
# Legacy script with repeated logic print("Area of
Rectangle:", 5 * 10) print("Perimeter of
Rectangle:", 2 * (5 + 10)) print("Area of
Rectangle:", 7 * 12) print("Perimeter of
Rectangle:", 2 * (7 + 12)) print("Area of
Rectangle:", 10 * 15) print("Perimeter of
Rectangle:", 2 * (10 + 15))
```

• Expected Output:

o Refactored code with a reusable function and no duplication.



Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

• Task: Use AI to analyze a Python script with nested loops and complex conditionals.

• Instructions:

o Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).

o Implement changes while keeping logic intact.

o Compare execution time before and after refactoring.

• Sample Legacy Code: # Legacy inefficient code names = ["Alice", "Bob", "Charlie", "David"] search_names = ["Charlie", "Eve", "Bob"] for s in search_names:

found = False for

n in names: if s ==

n:

found = True

if found: print(f"{s} is in the

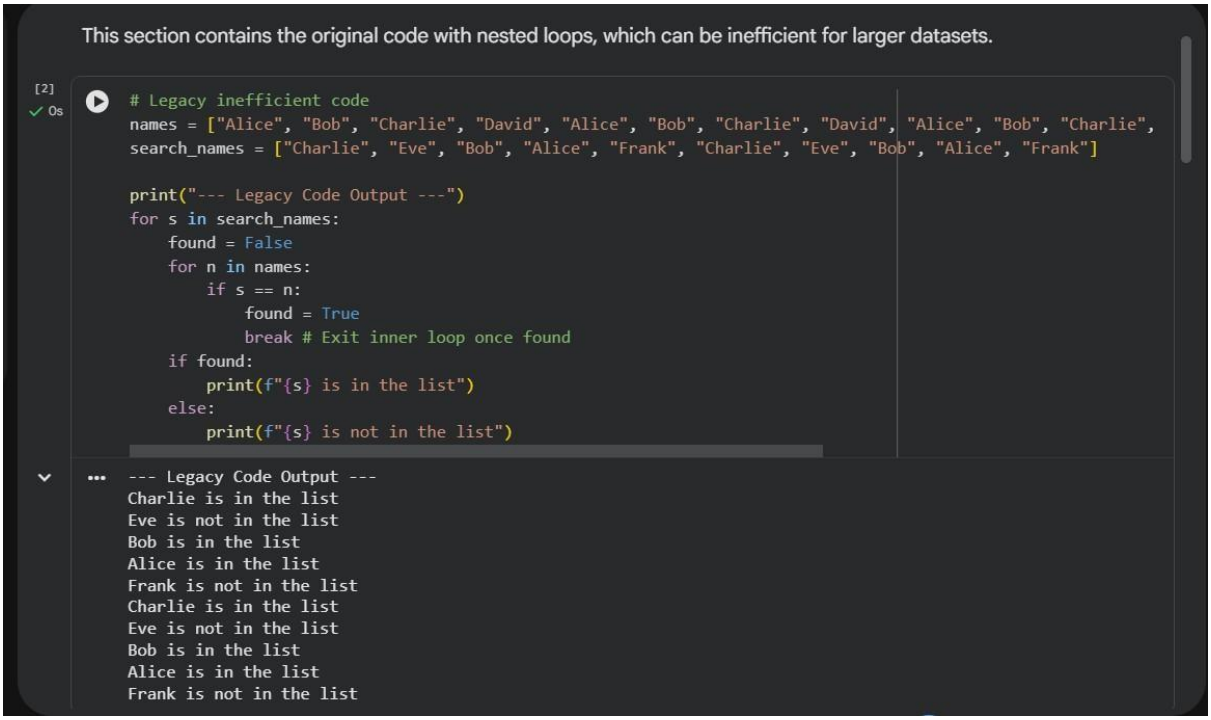
list") else: print(f"{s} is not

in the list")

• Expected Output:

o Optimized code using set lookups with performance

comparison table.



Optimized Code using Set Lookups

Here, I'm refactoring the code to use a set for `names`. Set lookups (checking if an item is `in` a set) have an average time complexity of O(1), making them significantly faster than list lookups (O(n)) for large collections.

[3] ✓ 0s

```
# Optimized code using set lookups
names_set = set(names) # Convert the list to a set once

print("\n--- Optimized Code Output ---")
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

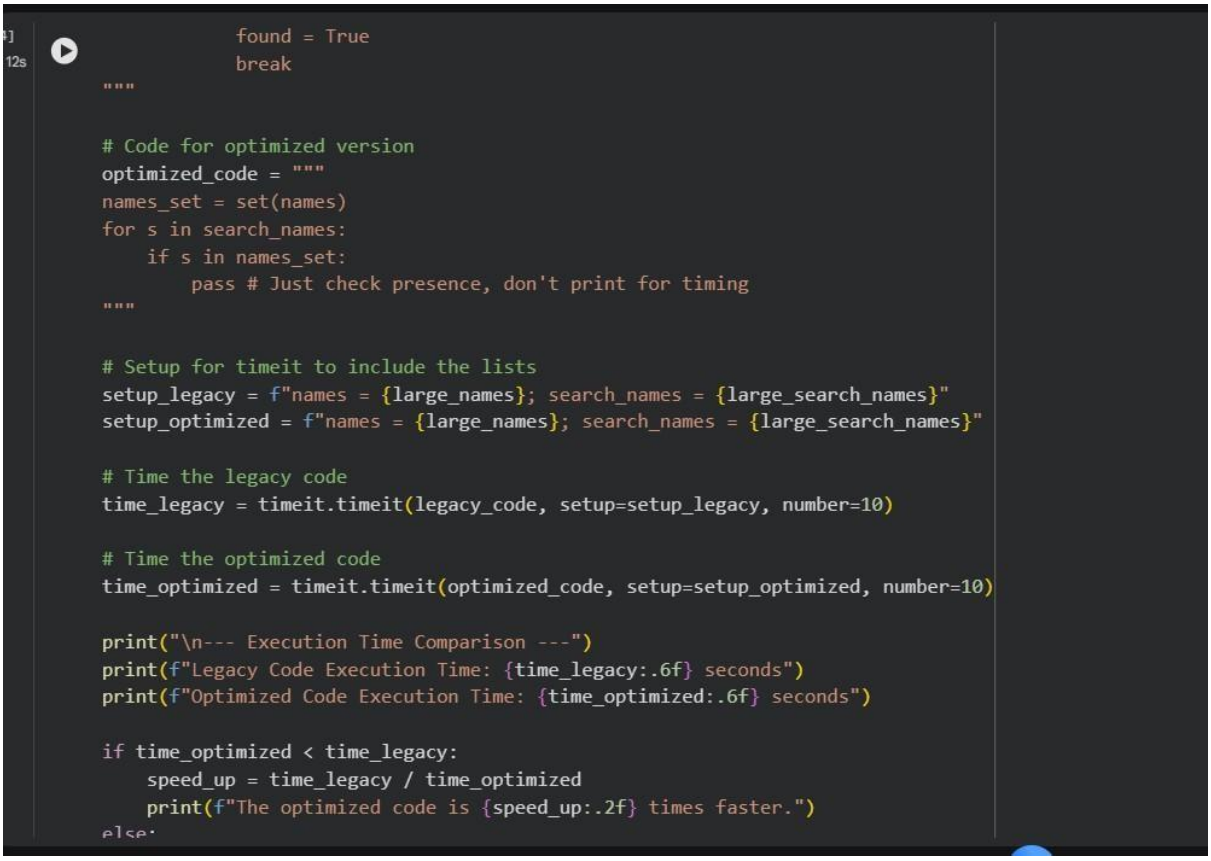
```
--- Optimized Code Output ---
Charlie is in the list
Eve is not in the list
Bob is in the list
Alice is in the list
Frank is not in the list
Charlie is in the list
Eve is not in the list
Bob is in the list
Alice is in the list
Frank is not in the list
```

Performance Comparison

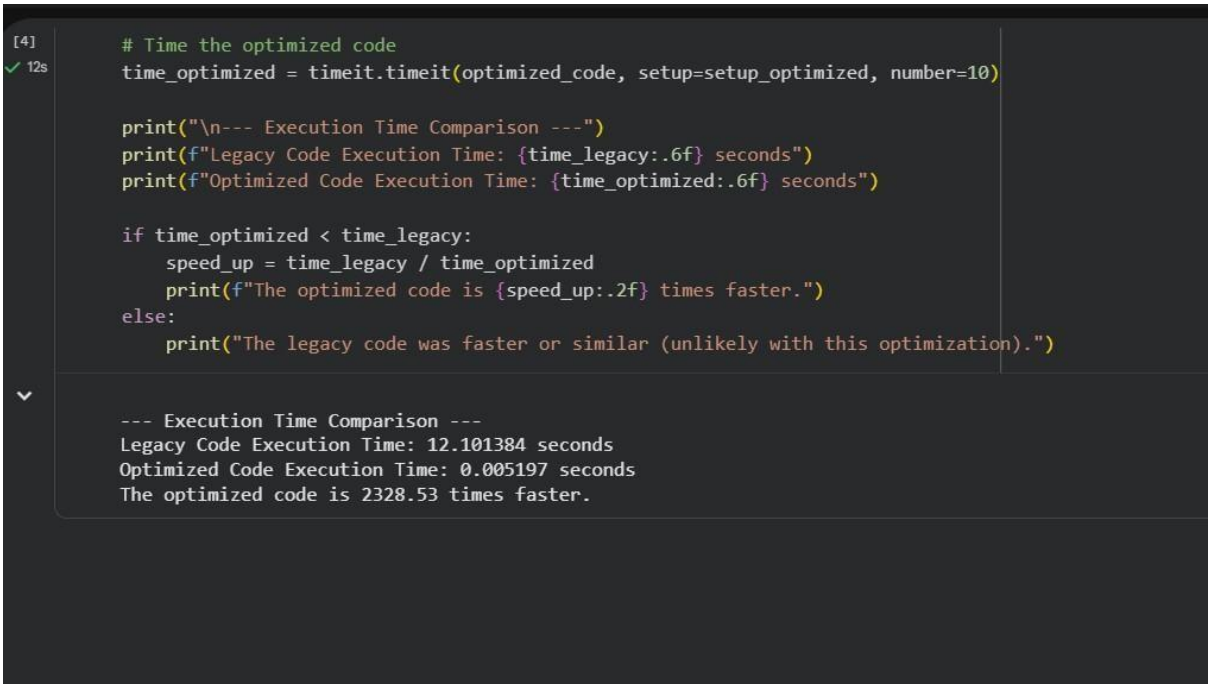
Now, let's compare the execution times of both the legacy and optimized versions using Python's `timeit` module. I'll use larger lists to highlight the performance difference more clearly.

[4] ✓ 12s

```
import timeit
import random
import string

# Generate larger lists for a clearer comparison
def generate_random_strings(num_strings, length=10):
    return [''.join(random.choices(string.ascii_letters, k=length)) for _ in range(num_strings)]

# Create a larger base list
large_names = generate_random_strings(10000) # 10,000 unique names
# Add some duplicates and also some names that will be searched
large_names.extend(generate_random_strings(2000))

# Create a large search list, some present, some not
large_search_names = generate_random_strings(5000) # 5,000 search names
# Ensure some search names are definitely in large_names
large_search_names.extend(random.sample(large_names, 1000))

# Code for legacy version
legacy_code = """
for s in search_names:
    found = False
    for n in names:
```



```
            found = True
            break
"""

# Code for optimized version
optimized_code = """
names_set = set(names)
for s in search_names:
    if s in names_set:
        pass # Just check presence, don't print for timing
"""

# Setup for timeit to include the lists
setup_legacy = f"names = {large_names}; search_names = {large_search_names}"
setup_optimized = f"names = {large_names}; search_names = {large_search_names}"

# Time the legacy code
time_legacy = timeit.timeit(legacy_code, setup=setup_legacy, number=10)

# Time the optimized code
time_optimized = timeit.timeit(optimized_code, setup=setup_optimized, number=10)

print("\n--- Execution Time Comparison ---")
print(f"Legacy Code Execution Time: {time_legacy:.6f} seconds")
print(f"Optimized Code Execution Time: {time_optimized:.6f} seconds")

if time_optimized < time_legacy:
    speed_up = time_legacy / time_optimized
    print(f"The optimized code is {speed_up:.2f} times faster.")
else:
```



```
[4]
✓ 12s
# Time the optimized code
time_optimized = timeit.timeit(optimized_code, setup=setup_optimized, number=10)

print("\n--- Execution Time Comparison ---")
print(f"Legacy Code Execution Time: {time_legacy:.6f} seconds")
print(f"Optimized Code Execution Time: {time_optimized:.6f} seconds")

if time_optimized < time_legacy:
    speed_up = time_legacy / time_optimized
    print(f"The optimized code is {speed_up:.2f} times faster.")
else:
    print("The legacy code was faster or similar (unlikely with this optimization).")

--- Execution Time Comparison ---
Legacy Code Execution Time: 12.101384 seconds
Optimized Code Execution Time: 0.005197 seconds
The optimized code is 2328.53 times faster.
```

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions: o Identify repeated or related logic

and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and

documented with docstrings.

• Sample Legacy Code:

Legacy script with inline repeated logic

price = 250 tax = price * 0.18 total = price

+ tax print("Total Price:", total) price =

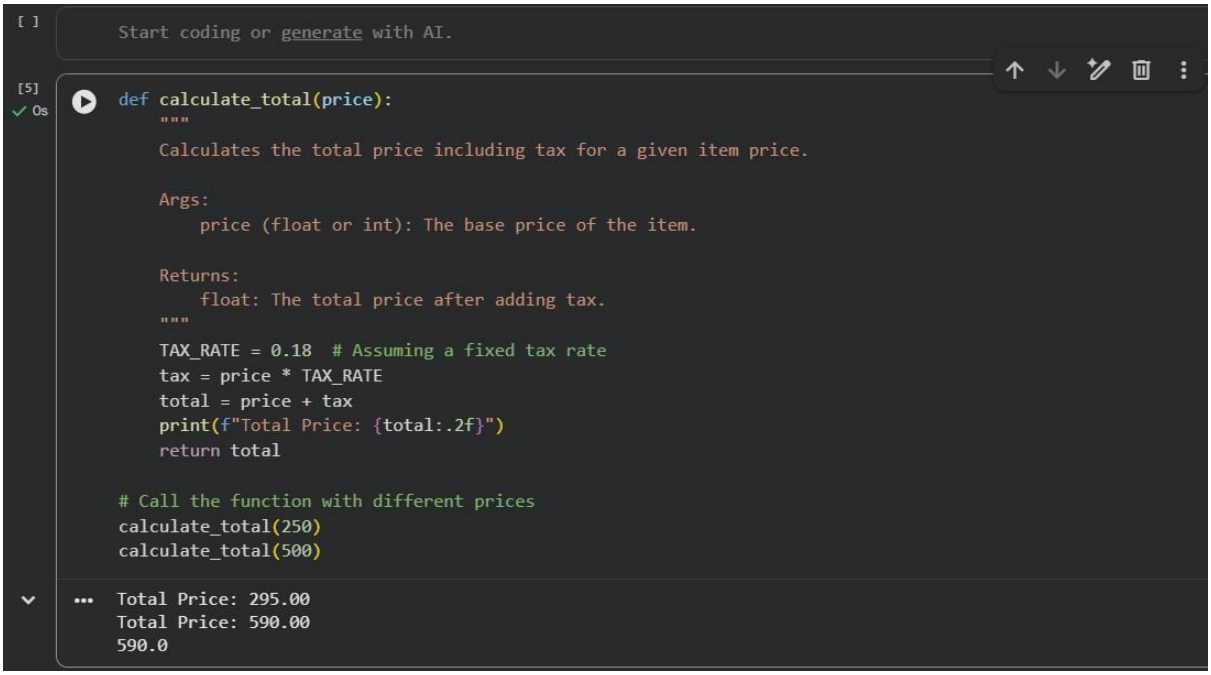
500 tax = price * 0.18 total = price + tax

print("Total Price:", total)

• Expected Output: o Code with a function calculate_total(price)

that can be

reused for multiple price inputs.



Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Use AI to identify and replace all hardcoded “magic numbers”

in the code with named constants.

• Instructions:

o Create constants at the top of the file.

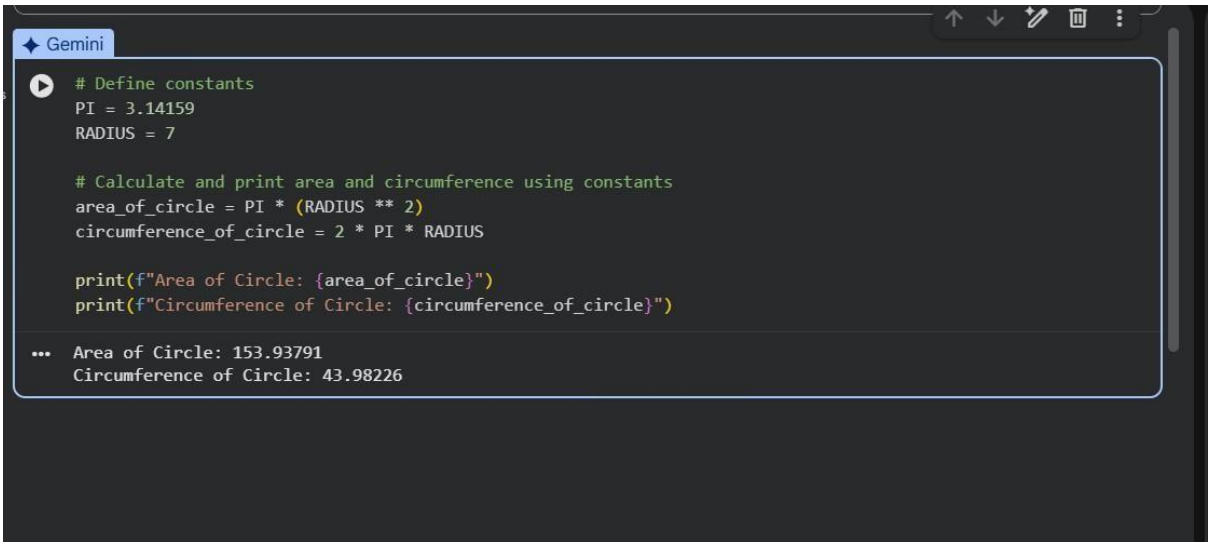
o Replace all hardcoded occurrences in calculations with these constants.

o Ensure the code remains functional and is easier to maintain.

• Sample Legacy Code: # Legacy script with hardcoded values print("Area of Circle:", 3.14159 * (7 ** 2)) print("Circumference of Circle:", 2 * 3.14159 * 7)

• Expected Output:

o Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.



• Task: Use AI to improve readability by renaming unclear variables and adding inline comments.

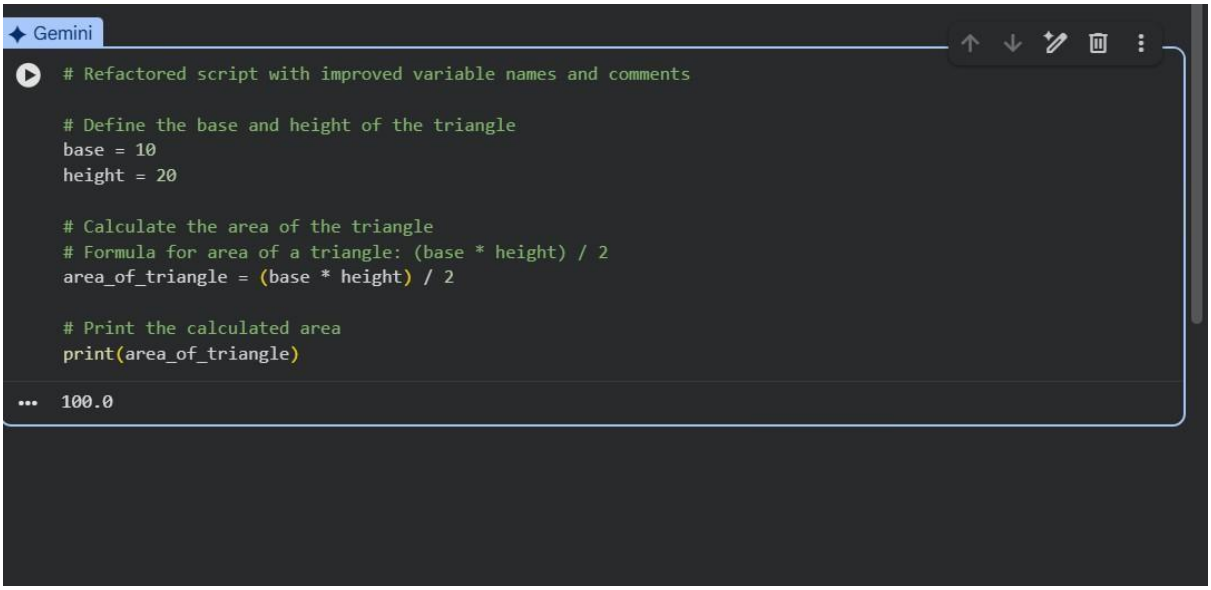
• Instructions:

o Replace vague names with meaningful ones. o Add comments where logic is not obvious. o Keep functionality exactly the same.

• Sample Legacy Code:

Legacy script with poor variable names a = 10 b = 20 c = a * b / 2 print(c)

• Expected Output: o Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.



Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else

logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function. o

Ensure grading logic remains unchanged.

• Code: marks =

85 if marks >=

90: print("Grade

A") elif marks >=

75:

print("Grade B")

else:

print("Grade C")

marks = 72 if marks

>= 90: print("Grade

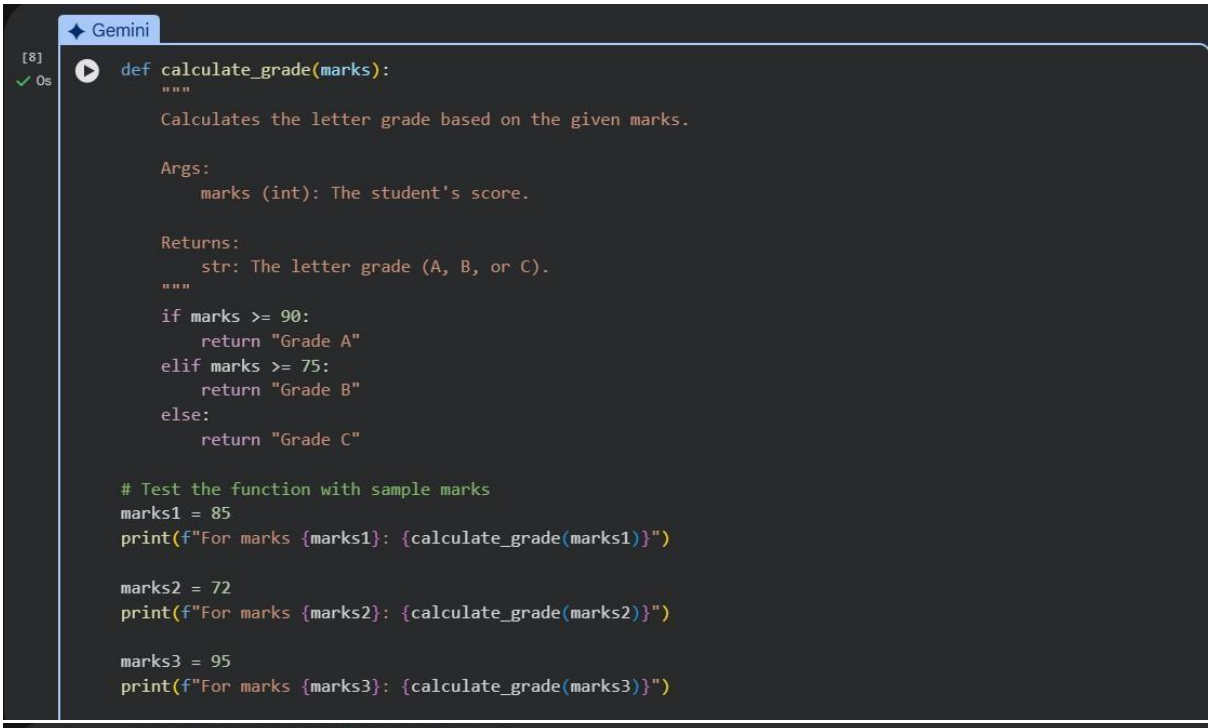
A") elif marks >=

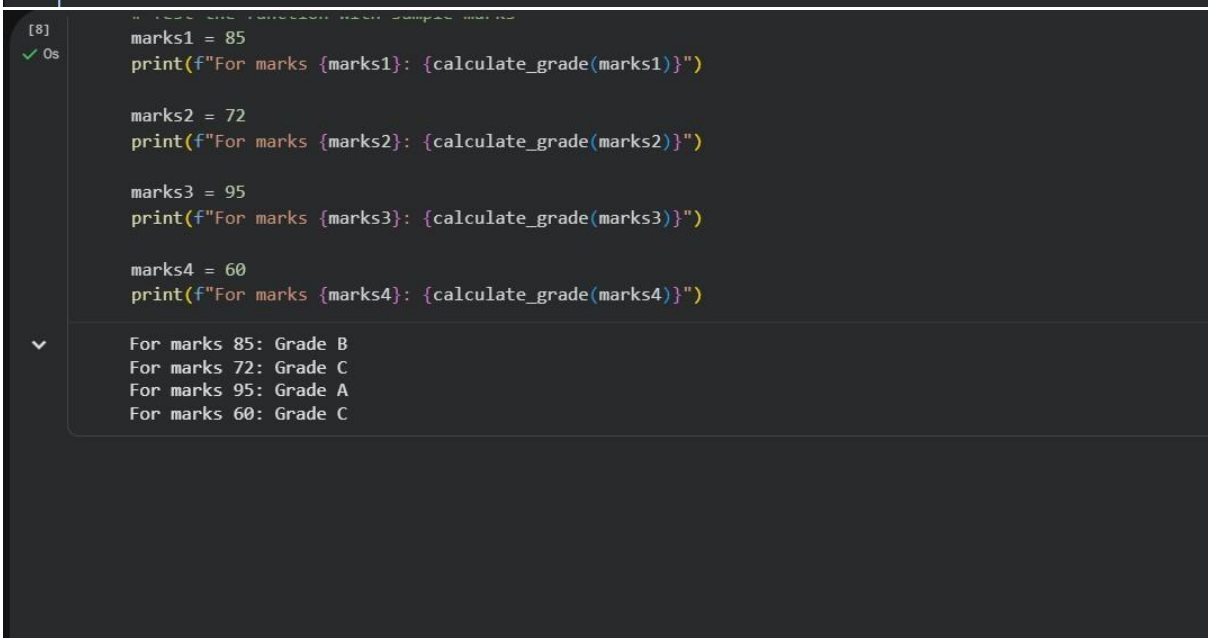
75: print("Grade

B")

else:

print("Grade C") • Expected Output: o A reusable function like calculate_grade(marks) with clean logic and docstring.





Task Description #7 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

• Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function. o Improve code readability without changing behavior.

- Sample Legacy Code: num = int(input("Enter

number: ")) square = num * num print("Square:", square)

- Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result().

```
[9]
✓ 5s
def get_input():
    """
    Prompts the user to enter an integer and returns it.

    Returns:
        int: The integer number entered by the user.
    """
    while True:
        try:
            num_str = input("Enter number: ")
            num = int(num_str)
            return num
        except ValueError:
            print("Invalid input. Please enter an integer.")

def calculate_square(number):
    """
    Calculates the square of a given number.

    Args:
        number (int): The number to be squared.

    Returns:
        int: The square of the number.
    """
    return number * number
```

```
[9]
✓ 5s
def display_result(original_number, squared_number):
    """
    Displays the original number and its square.

    Args:
        original_number (int): The number provided by the user.
        squared_number (int): The square of the original number.
    """
    print(f"The square of {original_number} is: {squared_number}")

# Main execution flow
if __name__ == "__main__":
    # Get input from the user
    input_number = get_input()

    # Calculate the square
    squared_value = calculate_square(input_number)

    # Display the result
    display_result(input_number, squared_value)

...  Enter number: 20
The square of 20 is: 400
```

Task Description #8 (Refactoring – Optimizing List Processing)

• Task: Use AI to refactor inefficient list processing logic.

• Instructions: o Ask AI to replace manual loops with list comprehensions or built-in functions. o Ensure output remains identical.

• Sample Legacy Code: nums = [1, 2, 3, 4, 5] squares = [] for n in nums: squares.append(n * n) print(squares) • Expected Output:

o Optimized version using list comprehension with improved readability